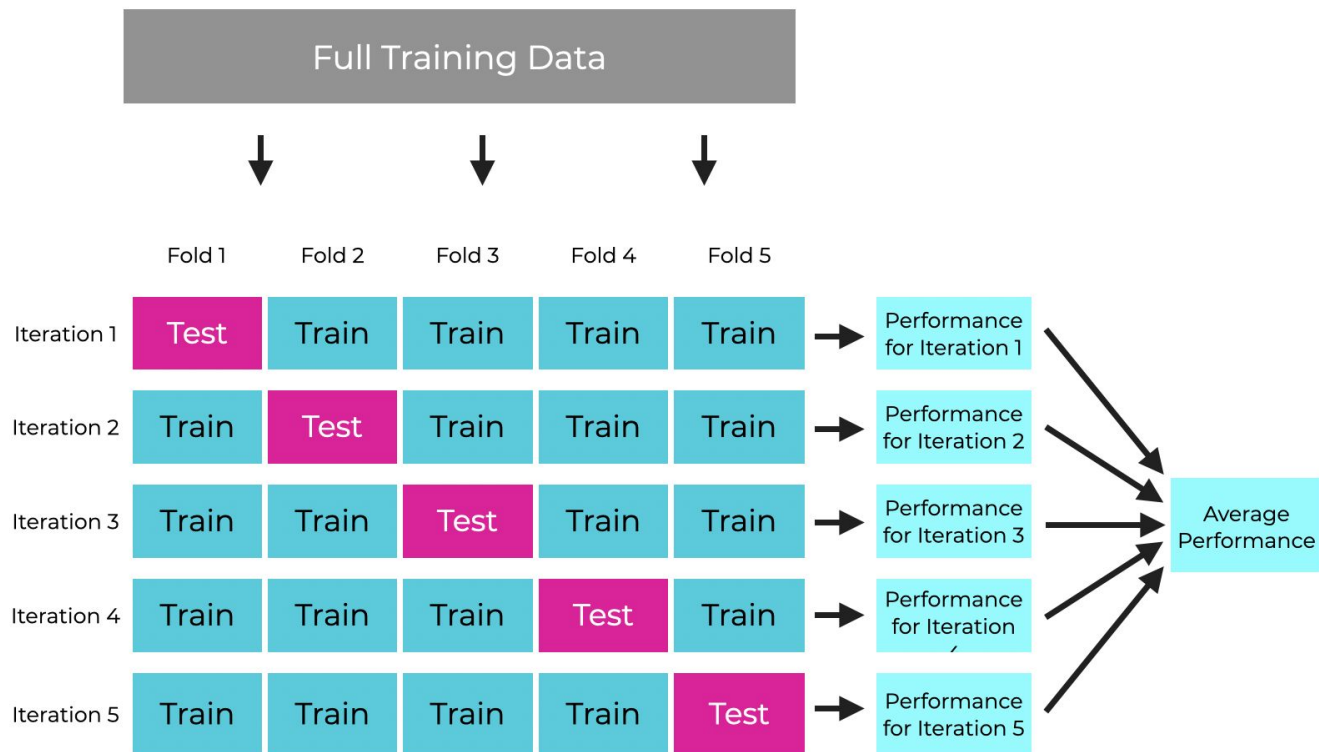


Cross-Validation Explained



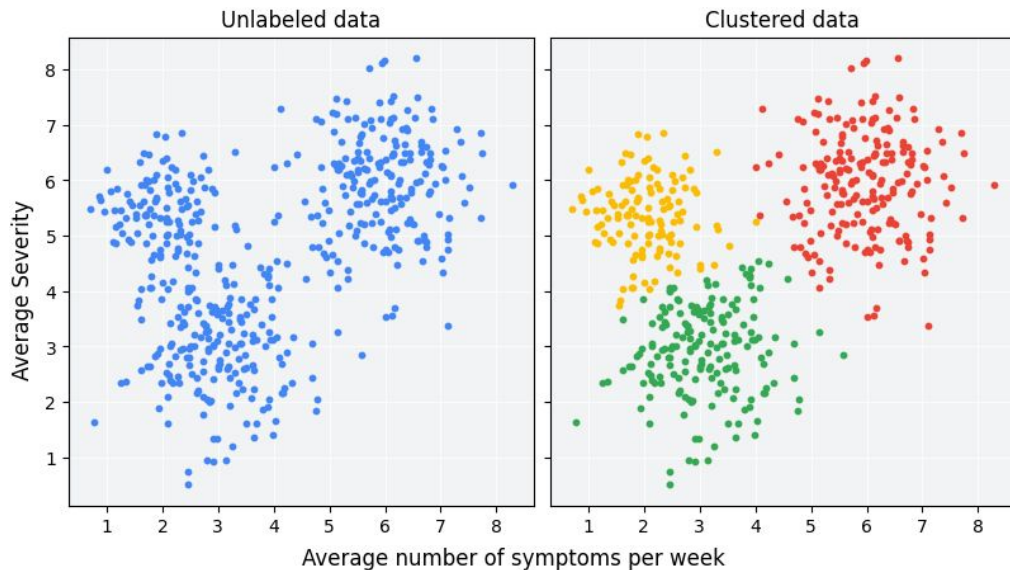
Unsupervised Learning I: Clustering

Instructor: Caryn Geady, PhD

Date: 9th Feb 2025

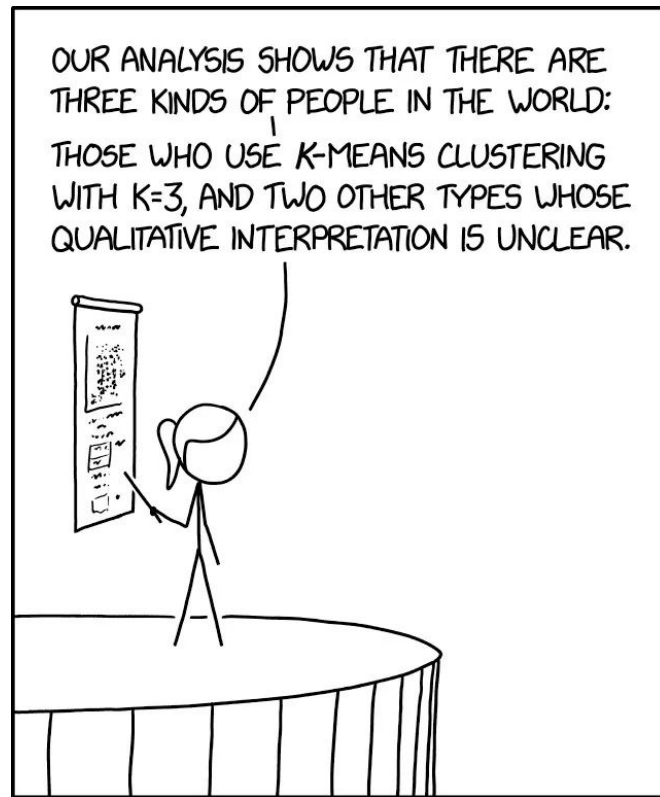
What is Clustering?

- **Unsupervised** machine learning technique designed to group **unlabeled examples** based on their **similarity** to each other



Today's Objectives

- Introduction to Clustering:
 - k-Means;
 - Hierarchical:
 - Agglomerative;
 - Divisive;
- Use Cases;
- Metrics for Evaluation:
 - Key Metrics: Silhouette Score, Dendrograms;
- Challenges:
 - Interpretation;
 - Determining the number of clusters



Clustering Algorithms

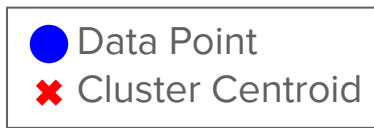
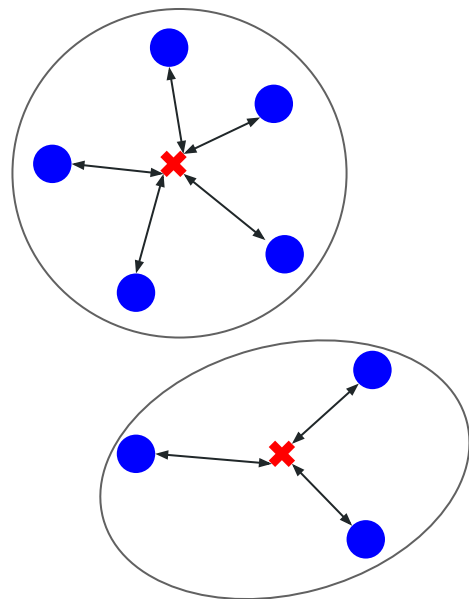
K-Means Clustering

Definition

- An **unsupervised** machine learning algorithm used for partitioning data into **K clusters**.
- Assigns data points to clusters based on their **distance** to the nearest cluster center (centroid).
- Iteratively optimizes cluster assignments to minimize intra-cluster variance (sum of squared distances).

Types of KMeans

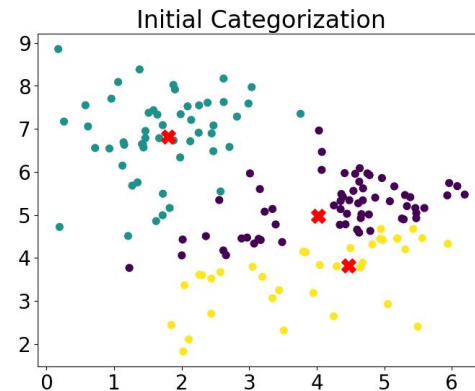
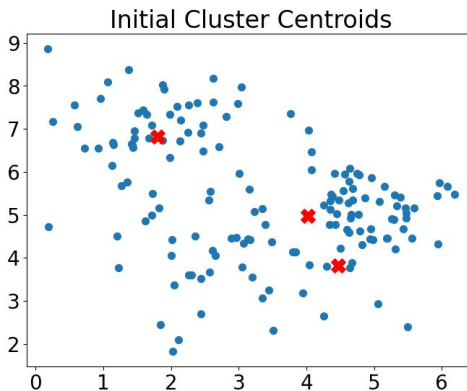
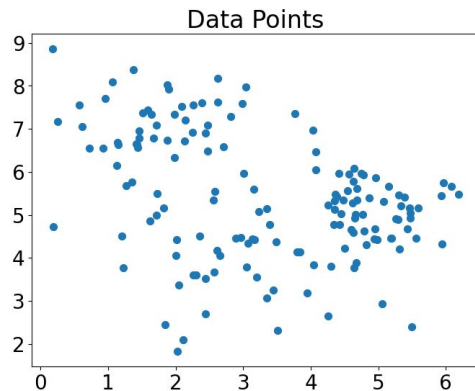
- Standard KMeans, KMeans++, Mini-Batch KMeans



K-Means Clustering

Algorithm:

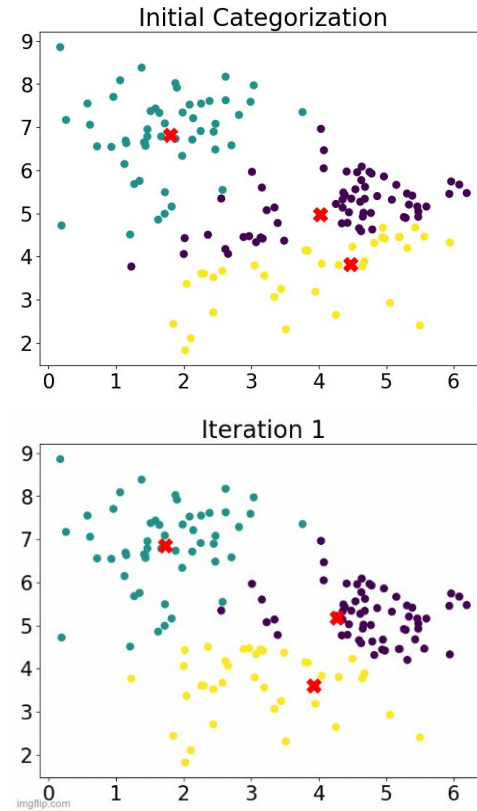
1. **Initialize centroids:** Randomly select k data points as the initial cluster centroids.
2. **Assign clusters:** Assign each data point to the nearest centroid based on a distance metric (e.g., Euclidean distance).



K-Means Clustering

Algorithm:

3. **Update centroids:** Calculate the mean of all data points in each cluster and update the centroids to these mean values.
4. **Repeat:**
 - Reassign data points to the nearest centroid.
 - Recalculate centroids.
 - Continue until centroids no longer change significantly or a maximum number of iterations is reached.
5. **Output:** Final cluster assignments and centroids.



K-Means Clustering in Practice

```
import numpy as np
from sklearn.cluster import KMeans
```

```
# Generate some synthetic data
```

```
np.random.seed(0)
```

```
X1 = np.random.normal(loc=[3, 4], scale=1.1, size=(50, 2))
```

```
X2 = np.random.normal(loc=[5, 5], scale=0.5, size=(50, 2))
```

```
X3 = np.random.normal(loc=[2, 7], scale=0.9, size=(50, 2))
```

```
X = np.vstack([X1, X2, X3])
```

```
# Fit the data using KMeans
```

```
k = 3
```

```
kmeans = KMeans(n_clusters=k, init='random', n_init=1, max_iter=1, random_state=1)
```

```
kmeans.fit(X)
```

The algorithm will run only once with the initial centroids

Random seed is fixed to 1 for reproducibility

The initial centroids are chosen randomly from the data points.

The algorithm will run only once with the initial centroids

The algorithm will try to find k clusters in the data.

K-Means Clustering

Advantages:

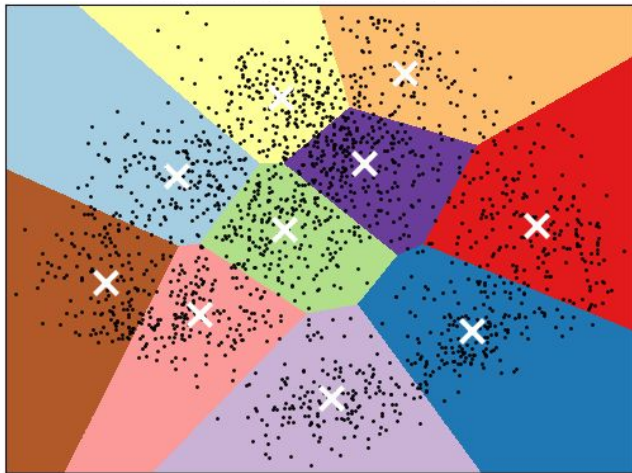
- Simple and fast for large datasets.
- Efficient for spherical and well-separated clusters.

Limitations:

- Requires predefining k .
- Sensitive to initial centroid placement.
- Struggles with non-spherical clusters and outliers.

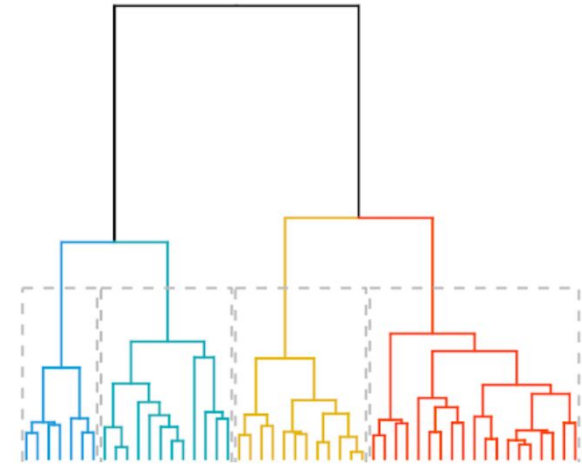
Applications:

- Customer segmentation, image recognition, and anomaly detection.



Hierarchical Clustering

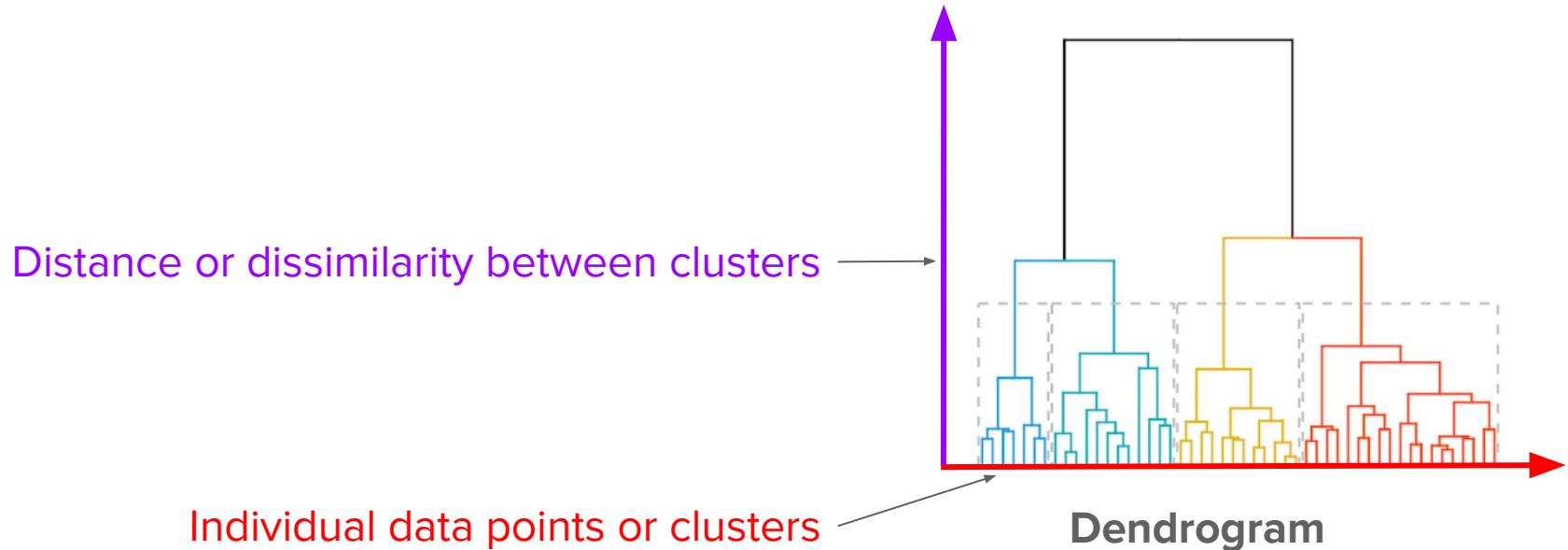
Definition: A clustering technique that builds a hierarchy of clusters using a tree-like structure (dendrogram).



Dendrogram

Hierarchical Clustering

Definition: A clustering technique that builds a hierarchy of clusters using a tree-like structure (dendrogram).

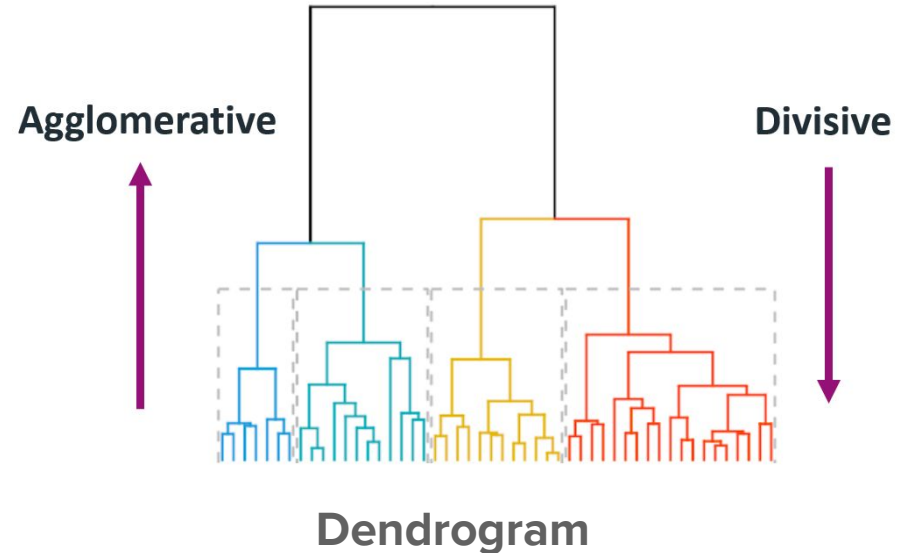


Hierarchical Clustering

Definition: A clustering technique that builds a hierarchy of clusters using a tree-like structure (dendrogram).

Types:

- **Agglomerative** (bottom-up): Starts with each data point as its own cluster and merges clusters iteratively.
- **Divisive** (top-down): Starts with all points in a single cluster and splits them iteratively.



Hierarchical Clustering

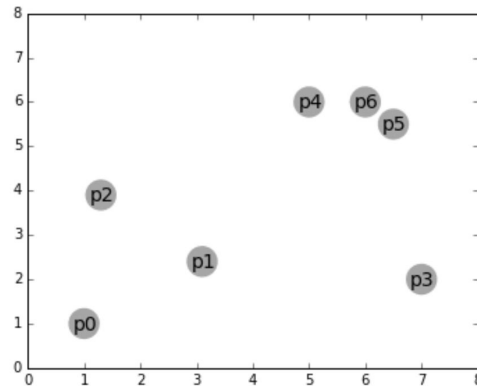
Algorithm (Agglomerative)

1. Initialization

- Treat each data point as its own cluster.
- Start with N clusters for N data points.

2. Compute Pairwise Distances

- Calculate distances (e.g., Euclidean) between all clusters.
- Use a **linkage method** to determine cluster distances:
 - **Single Linkage**: Minimum distance between clusters.
 - **Complete Linkage**: Maximum distance between clusters.
 - **Average Linkage**: Mean distance between clusters.
 - **Centroid Linkage**: Distance between cluster centroids.



Hierarchical Clustering

Algorithm (Agglomerative)

3. Merge Closest Clusters

- Identify the two clusters with the smallest distance.
- Merge them into a new, larger cluster.

4. Update Distance Matrix

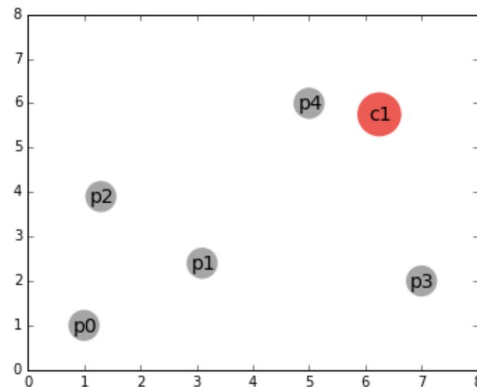
- Recalculate distances between the new cluster and all remaining clusters using the chosen linkage method.

5. Repeat

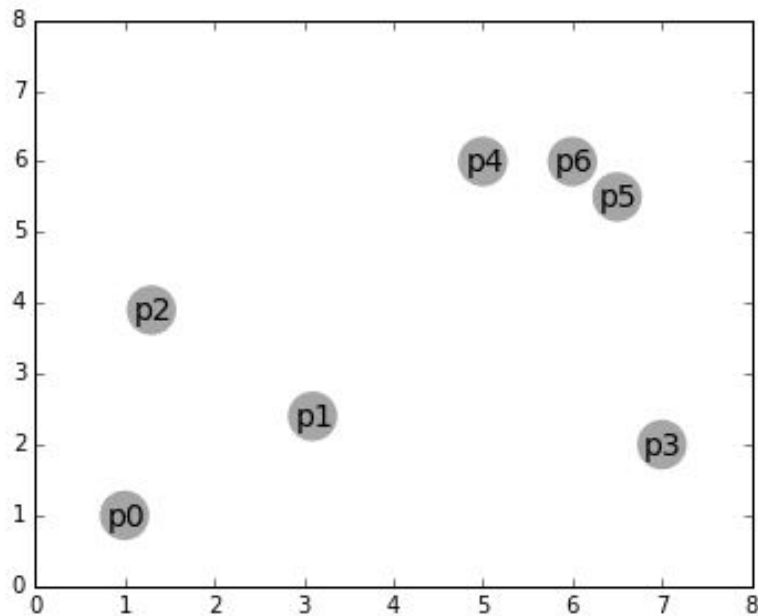
- Continue merging clusters iteratively until only one cluster remains.

6. Dendrogram (Visualization)

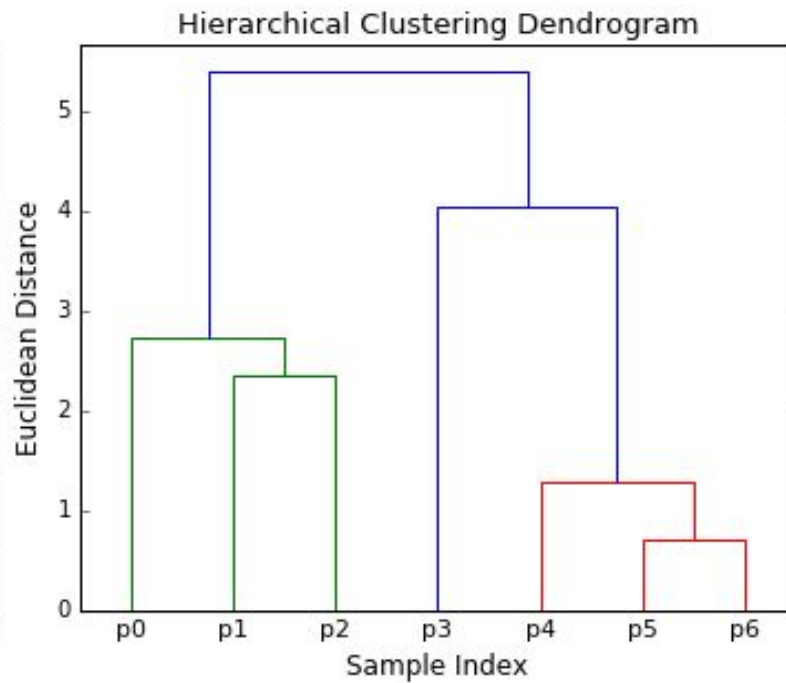
- Cutting the dendrogram at a specific height provides the final clusters.



Hierarchical Clustering



Data Points



Dendrogram

Hierarchical Clustering in Practice

```
import numpy as np
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.preprocessing import StandardScaler

# Generate sample data
np.random.seed(0)
X1 = np.random.normal(loc=[3, 4], scale=1.1, size=(10, 2))
X2 = np.random.normal(loc=[5, 5], scale=0.5, size=(10, 2))
X3 = np.random.normal(loc=[2, 7], scale=0.9, size=(10, 2))
X = np.vstack([X1, X2, X3])

# Standardize the data
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Perform hierarchical clustering
Z = linkage(X_scaled, method='complete',
metric='euclidean',
optimal_ordering=True)
```

Uses complete linkage, where the distance between clusters is the maximum distance between any two points in the clusters.

Uses Euclidean distance to measure the distance between data points.

Orders the leaves of the dendrogram to minimize the distances between successive leaves, making the dendrogram easier to interpret.

Hierarchical Clustering

Advantages:

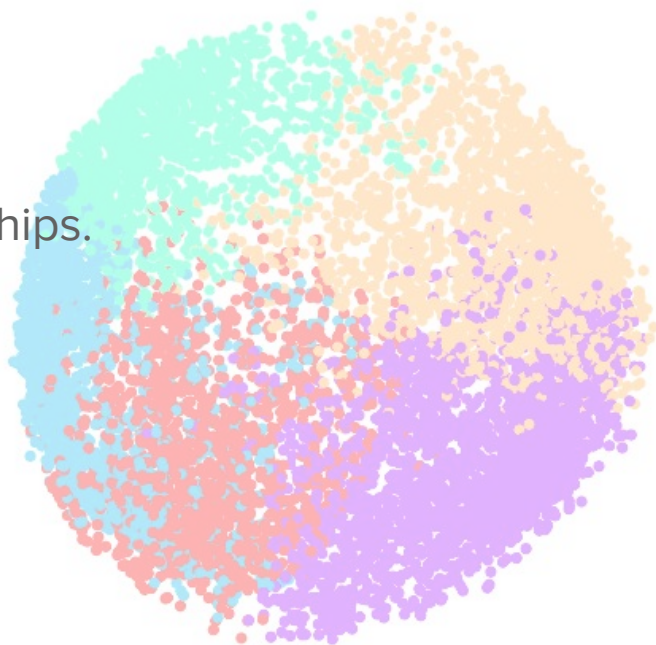
- No need to predefine the number of clusters.
- Produces a clear hierarchy for cluster relationships.

Limitations:

- Computationally expensive for large datasets.
- Sensitive to noise and outliers.

Applications:

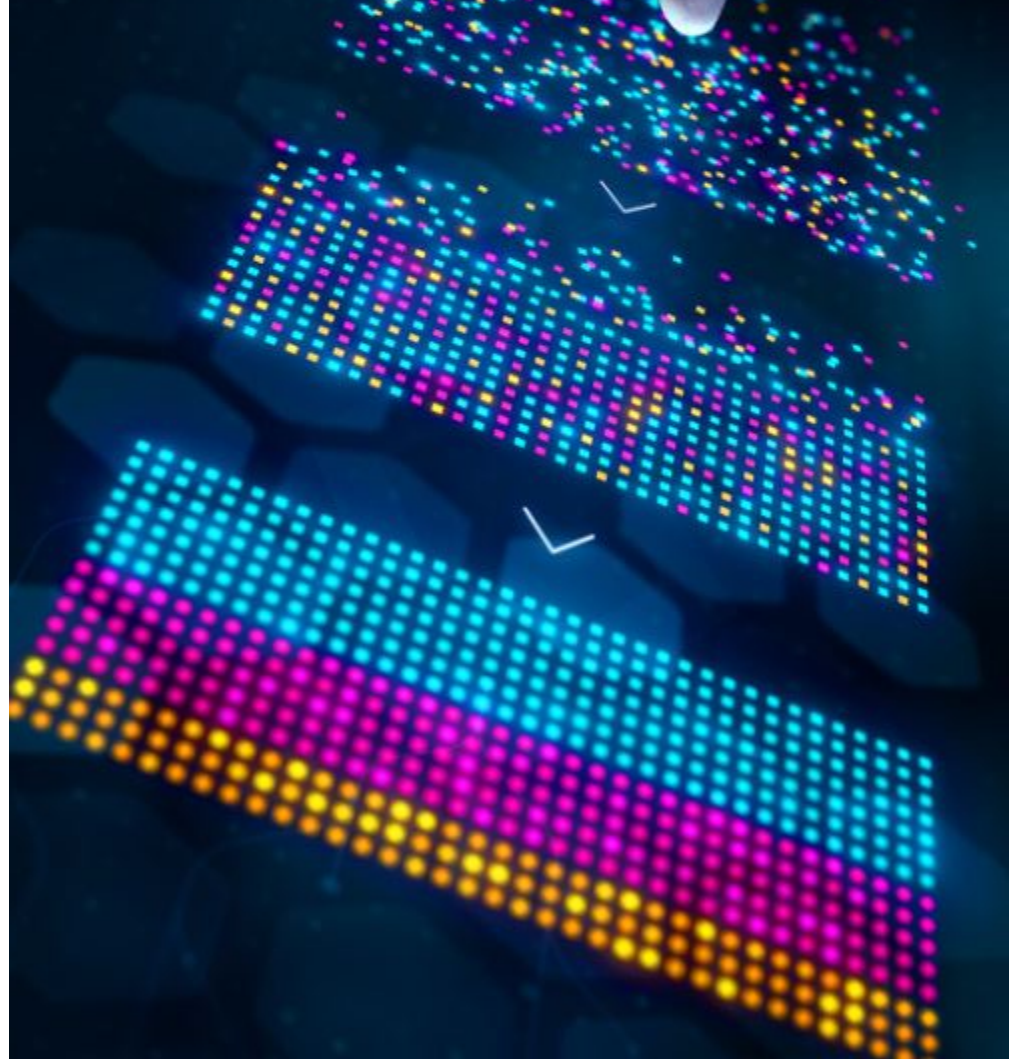
- Gene expression analysis, document clustering, and phylogenetic trees.



Use Cases

Clustering Use Cases

- **Clinical bioinformatics:**
 - Patient stratification (e.g., grouping patients with similar disease patterns).
 - Gene expression analysis (e.g., clustering genes with similar expression profiles).
- **Other domains:**
 - Market segmentation in business.
 - Image segmentation in computer vision.



Metrics for Evaluation

Silhouette Score



Definition

- A metric to evaluate the quality of clusters in clustering algorithms;
- Measures how well each data point fits within its assigned cluster vs. others;

Calculation

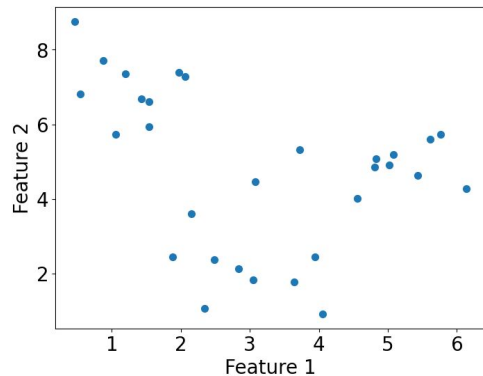
- **For each data point i:**
 - Compute **a(i)**: Average distance to all other points in the same cluster;
 - Compute **b(i)**: Average distance to all points in the nearest neighboring cluster.
- **Silhouette score for i (range [-1, 1]):**

$$S(i) = \frac{b(i) - a(i)}{\max(b(i), a(i))}$$

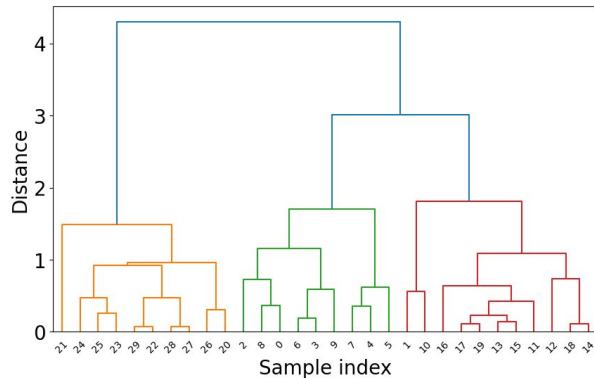
Silhouette Score

In Practice

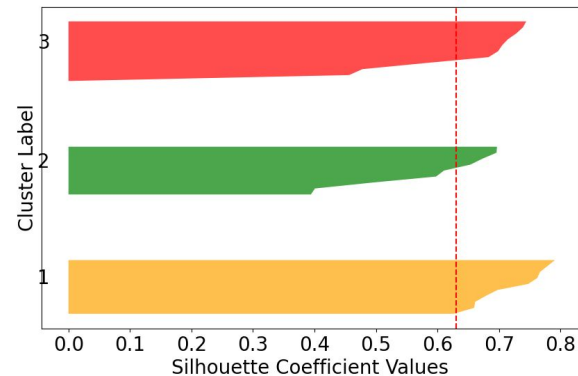
- Compare score for different values of k to choose optimal number of clusters;
- Visualize with a silhouette plot.



Data Points



Dendrogram



Silhouette Plot

Silhouette Score

Advantages

- **Interpretability:** Intuitive and easy to understand;
- **Applicability:** Works with any clustering algorithm and distance metric;
- **Global Evaluation:** Both individual point-level and overall cluster quality.

Limitations

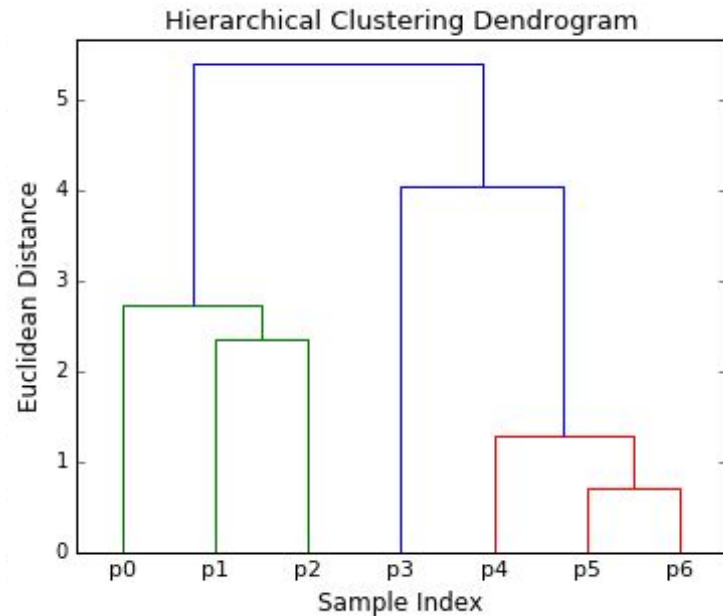
- **Computationally Expensive:** For large datasets, esp. hierarchical clustering.
- **Cluster Shapes:** Assumes well-separated, convex clusters; may mislead for non-convex shapes.
- **Dependence on Distance Metric:** Quality depends on the choice of distance measure.

Dendrograms



Interpretation

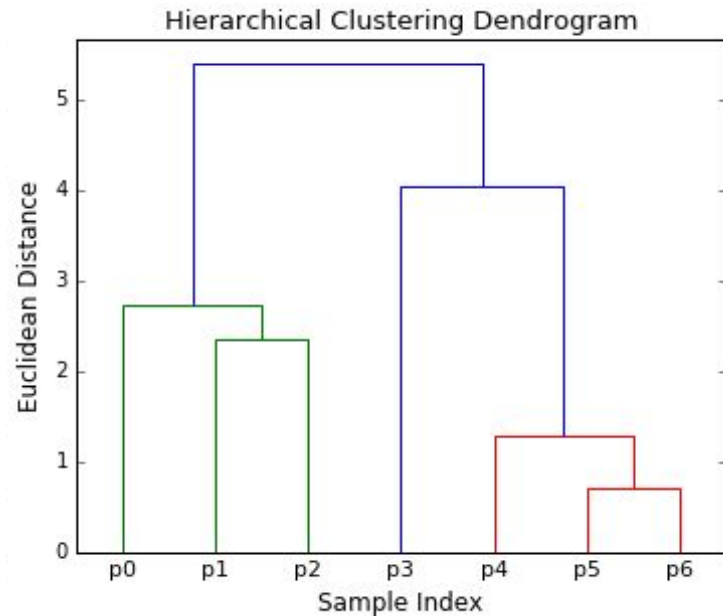
- Horizontal lines represent merges; their height shows the distance between merged clusters.
- The longer the vertical branch, the less similar the clusters.
- Decide on the number of clusters by cutting the tree at a specific height.
- Lower cut-offs = more clusters; higher cut-offs = fewer clusters.



Dendrogram

Dendrograms

- **Understanding Relationships:**
 - Provides insights into how data points are grouped across different levels of granularity.
 - Reveals nested cluster structures that flat clustering methods like K-Means cannot show.
- **Selecting Number of Clusters:**
 - Visualize potential clusters and identify natural groupings based on branch lengths and cut-offs.
- **Outlier Detection:**
 - Outliers may appear as single branches that don't merge with others until very high levels.

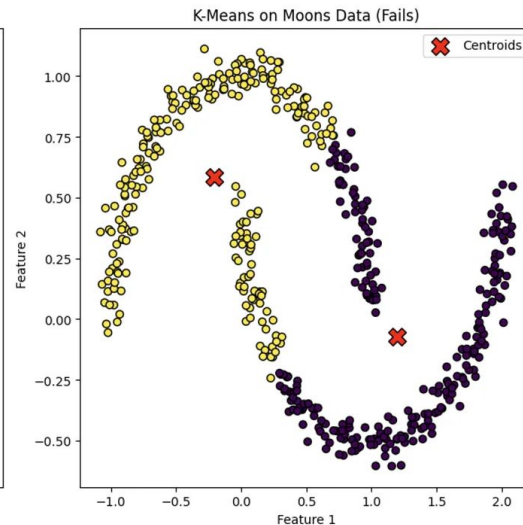
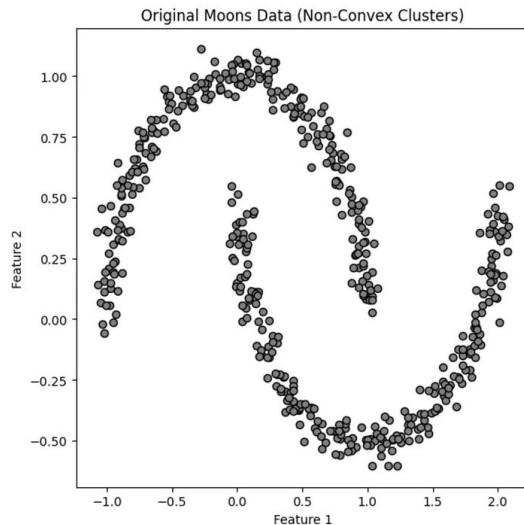
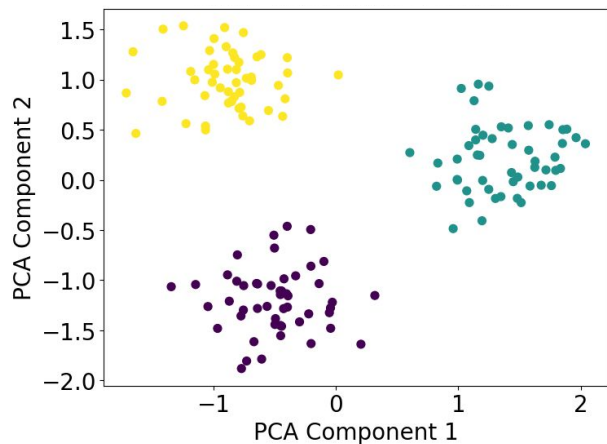


Dendrogram

Challenges in Clustering Analysis

Interpreting Clusters

- **Challenges:** Clusters are not always meaningful or biologically relevant.
- **Examples:**
 - Batch effects;
 - Non-convex data



Determining the Number of Clusters

Methods

- **Silhouette Score** (previous section)
- **Elbow Method**
 - **Definition:** Plotting variance explained vs. number of clusters k .
 - **Interpretation:** The "elbow point" (where the curve bends) indicates the optimal number of clusters, balancing explained variance and model simplicity.
- **Gap Statistic**
 - **Definition:** Compare within-cluster dispersion to that of random reference data.
 - **Interpretation:** Optimal k is where the gap between the clustering result and reference data is largest, indicating significant structure in the data.

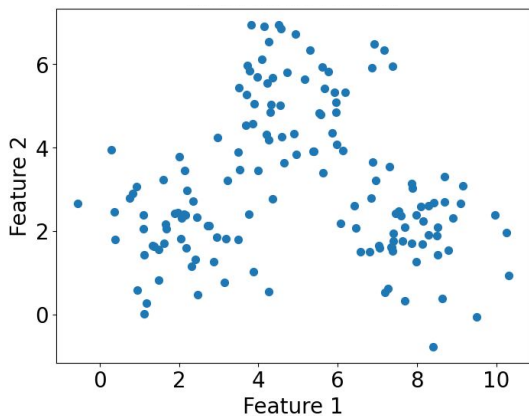
Challenge → Different methods may give different optimal k values!

Determining the Number of Clusters

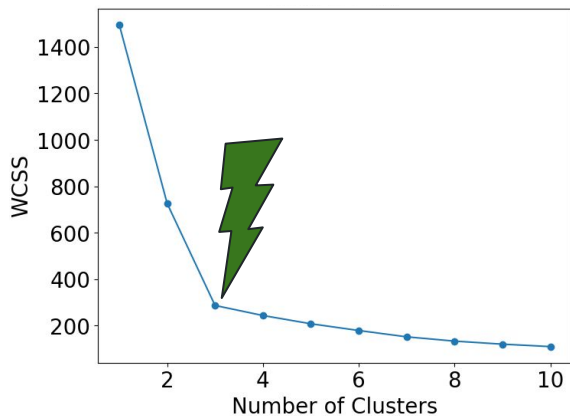


In Practice

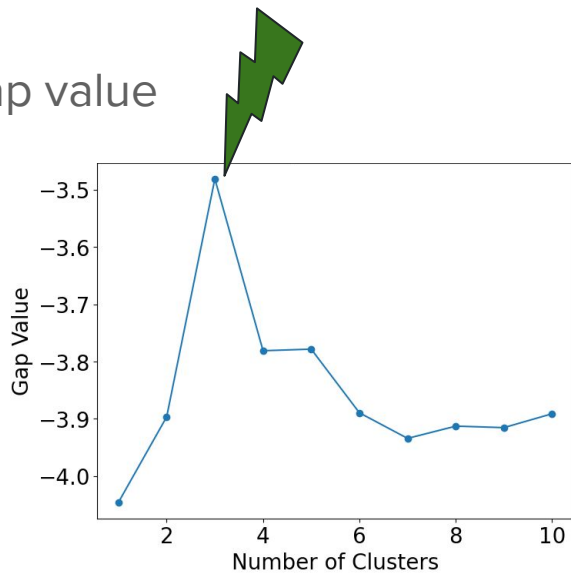
- **Optimal k** where curve bends (elbow plot) or peak gap value



Data Points



Elbow Plot



Gap Value

Summary and Q&A

Summary – Algorithms



Feature	KMeans Clustering	Hierarchical Clustering
Definition	Partitions data into K clusters by optimizing centroids.	Builds a tree-like hierarchy of clusters (dendrogram).
Approach	Iterative optimization.	Agglomerative (bottom-up) or Divisive (top-down).
Number of Clusters	Requires predefined K .	Does not require specifying K beforehand.
Cluster Shape	Assumes spherical, equal-sized clusters.	Works well for clusters of arbitrary shapes and sizes.
Initialization	Centroids are randomly initialized; KMeans++ improves this.	No initialization required.

Summary – Algorithms

Feature	KMeans Clustering	Hierarchical Clustering
Result	Flat partition of clusters.	Nested clusters in a hierarchical tree structure.
Interpretability	Moderate; centroids summarize each cluster.	High; dendrogram provides detailed visual cluster relationships.
Strengths	Simple, fast, and efficient for large datasets.	No need for K, better suited for small/medium datasets.
Limitations	Requires K, sensitive to initialization and outliers.	Computationally expensive for large datasets.

Summary – Evaluation and Challenges

Aspect	Evaluation Metric	Challenges
Cluster Quality	- Silhouette Score: Measures cohesion and separation of clusters.	- Interpreting whether high/low scores reflect true data structure; - Sensitive to noise or outliers.
Number of Clusters	- Elbow Method: Identifies optimal k by finding the "elbow" in an inertia plot; - Gap Statistic: Compares clustering with random data; - Dendrogram: Visualizes hierarchical relationships.	- No single method works universally for all datasets; - Ambiguity in choosing "elbow" or interpreting dendrogram cuts.

Questions?

